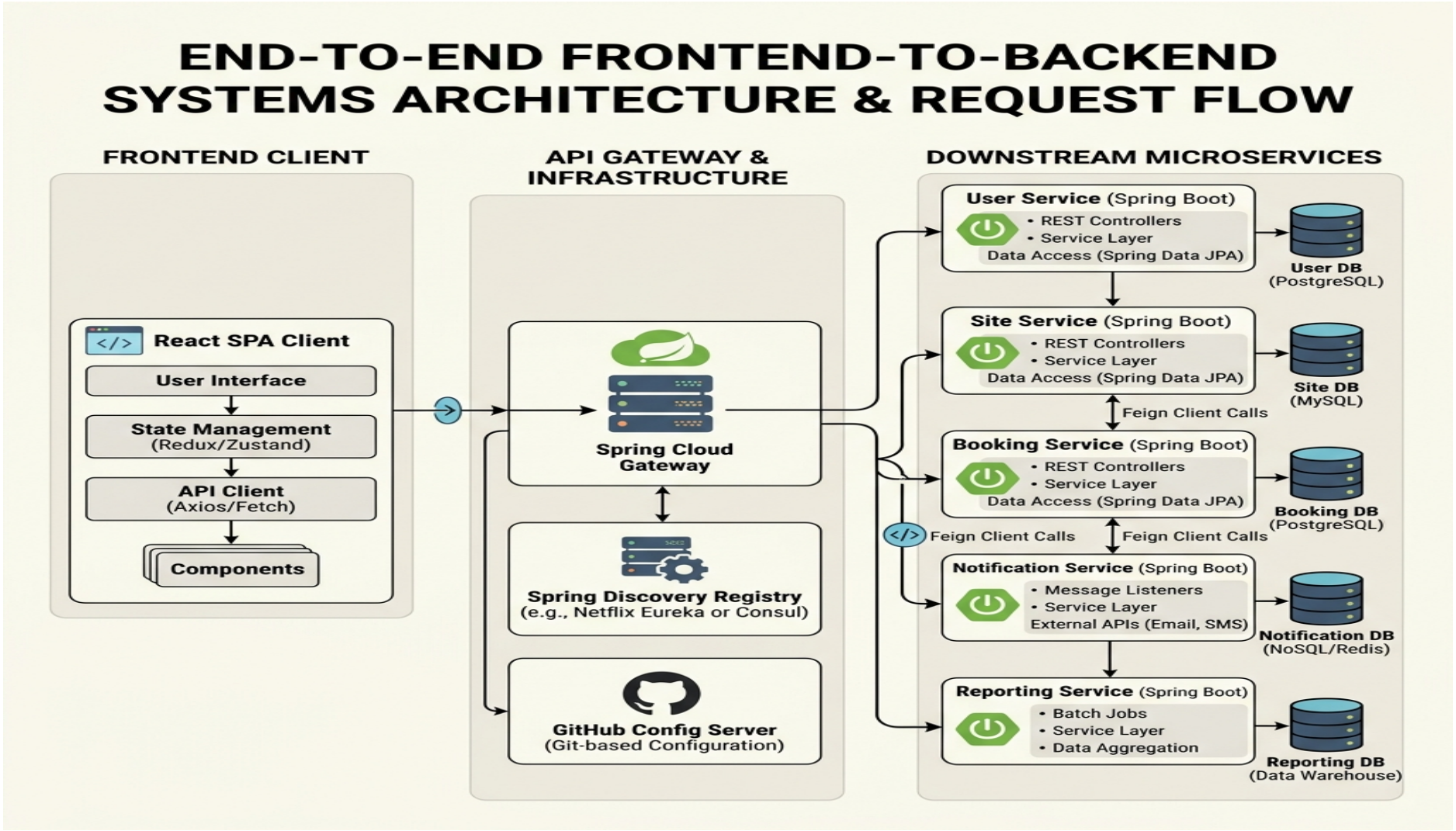
Tourism Government — Unified Systems Request Flow Chart

Production End-to-End Dynamic Request Journey & Architecture Mapping



Chapter 1: Verified 11-Service Port & Database Directory

Absolute, 100% accurate system directory mapped straight from workspace application.properties configuration profiles.

No.	Spring Service Name	Verified Port	MySQL isolated Schema	Core Systems Responsibility & Architectural Functionality
1	**USER-SERVICE**	**1111**	UserDB	Authenticates identity credentials, user registers, and roles checking (Tourist, Admin).
2	**TOURIST-SERVICE**	**2020**	touristdb	Manages comprehensive tourist profiles and records history logs.
3	**SITE-SERVICE**	**3030**	sitedb	Maintains registry of compliance histories of historical tourism site resources.
4	**EVENTBOOKING-SERVICE**	**4040**	eventbookingdb	Schedules heritage programs, manages events, and registers visitor booking bookings.
5	**PROGRAM-SERVICE**	**5050**	programdb	Manages tourism promotional initiatives, national campaigns, and budgets.
6	**COMPLIANCE-SERVICE**	**6060**	compliancedb	Performs strict compliance validation on heritage sites against policies.
7	**NOTIFICATION-SERVICE**	**7070**	notificationdb	Observes alerts queue, logs messages, and sends automated emails using JavaMailSender.
8	**REPORT-SERVICE**	**9090**	reportdb	Aggregates multithreaded dynamic lists from active services via OpenFeign client mappings.
9	**ConfigServer**	**8181**	Local / Git Repository	Serves properties files on port 8181 dynamically from GitHub to active microservices.
10	**EurekaServer**	**8761**	In-Memory Registry	Central dynamic lookups catalog. Translates application names to active IPs and Ports.
11	**GatewayAPI**	**8383**	In-Memory Router	Spring Cloud Routing Gateway port 8383. Performs JWT token verification and filters requests.

Chapter 2: Production End-to-End Tracing Workflow

How a client request travels across the frontend SPA and the decoupled Spring Cloud cluster.

Phase 1: Startup Configurations Fetch
All 9 active services (User, Tourist, Site, Event, Program, Compliance, Notification, and Reporting services) query the central **Config Server (Port 8181)** during bootstrap to dynamically load their port variables and MySQL database strings straight from GitHub.

Phase 2: React SPA Axios Request (Vite + Tailwind on Port 5173)
The React Client triggers an Axios REST call to request data. An Axios interceptor automatically attaches a dynamic **JWT (JSON Web Token)** security header to authenticate the user's role.

Phase 3: API Gateway Load-Balanced Routing (Port 8383)
All REST calls hit the Spring Cloud Gateway. The Gateway extracts the JWT, verifies its signature, queries the **Eureka Registry (Port 8761)** to find active microservice IP/Port mappings, and routes the request to the target service.

Phase 4: Service-to-Service Dynamic OpenFeign Hop
If the REPORT-SERVICE is called, it verifies that the user is not a tourist (throwing HTTP 403 otherwise). It then initiates dynamic, memory-mapped HTTP requests to user-service using userClient.getUserById(id) and site-service using siteClient.getAllSites() to aggregate compile metrics.

Phase 5: isolated Database-per-Service Persistence
Each service handles its own writes to its isolated database schema (e.g. UserDB, reportdb) using Spring Data JPA, guaranteeing that schema modifications in one system never break downstream logic.